Configurable log layouts and parser decisions

Quaestio layout support update completed October 7 2026. This guide supplements the original MVP implementation report; the original report describes the earlier baseline.

1 What is supported now

Quaestio can analyze many different single-line log layouts without changing Python source. A reusable JSON profile tells it how to extract timestamp, severity, source, and message. Text profiles use named regex groups; JSON profiles map field names and nested paths. Both produce the same LogEntry objects consumed by the existing analyzer.

This is configurable parsing, not a claim that every possible file can be interpreted automatically. Records still need a timestamp and message. A missing source can default to unknown; severity can default only when explicitly configured. Multiline stack traces, JSON arrays, binary formats, and logs needing cross-record context remain unsupported.

The old text and NDJSON parsers keep their existing behavior. New --format auto recognizes several common grammars and locks to the first valid grammar in a stream. --format json is a convenient alias for the existing NDJSON input contract, not whole-file JSON arrays. No runtime dependency or analysis-result schema was added.

2 Try the supplied examples

From the project directory, activate the environment and run:

```
source .venv/bin/activate
quaestio analyze examples/bracketed.log --format auto
quaestio analyze examples/custom-pipe.log \
  --parser-config examples/layouts/custom-pipe.json
quaestio analyze examples/nested.ndjson \
  --parser-config examples/layouts/nested-json.json
```

The custom pipe example has four entries: one INFO, one WARNING, and two ERROR. Its ERROR rate is 50%; Connection refused is the most frequent error with count two. The nested JSON example has three entries and two errors, for an ERROR rate of 66.7%. The bracketed example counts FATAL as CRITICAL, which remains separate from ERROR rate.

Profiles also work with stdin, filtering, exports, and invalid-record skipping:

```
cat examples/custom-pipe.log | quaestio analyze - \
  --parser-config examples/layouts/custom-pipe.json --output json
quaestio analyze examples/custom-pipe.log \
  --parser-config examples/layouts/custom-pipe.json --source database --output csv
quaestio analyze examples/nested.ndjson \
  --parser-config examples/layouts/nested-json.json \
  --output json --output-file report.json
```

The profile chooses the input format. Usually omit --format when supplying a profile. An explicitly different format is rejected. For example, --format auto cannot be combined with a

regex profile. json and ndjson are equivalent for this compatibility check.

3 Define your own text layout

Suppose your application writes severity first, then service, then a day/month/year date:

```
severe | database | 07/10/2026 10:30:05 | Connection refused
```

Save the following as a JSON profile. A complete example already exists at examples/layouts/custom-pipe.json:

```
{
  "format": "regex",
  "pattern": "(?P<level>[^|]+)\\|(?P<source>[^|]+)\\|(?P<timestamp>[^|]+)\\|\\s*(?P<message>.+)",
  "timestamp_format": "%d/%m/%Y %H:%M:%S",
  "level_map": {"severe": "ERROR"}
}
```

A named group has the form (?P<field>expression). The name tells Quaestio which field the captured text represents; the expression describes which characters belong to it. Groups can appear in any order, and literal labels, brackets, delimiters, or extra noncaptured fields can appear around them. The whole physical line must match.

Required groups are timestamp, level, and message. Instead of timestamp, a profile may capture both date and time, which are joined with one space before date conversion. Source is optional because the default profile settings supply unknown. Level can be omitted only when defaults explicitly contains a recognized level. Timestamp and message cannot be invented through defaults. Additional named groups are ignored.

JSON requires escaped backslashes: \\s represents regex \s, and \\| represents a literal pipe. To learn or debug a profile, start from the checked-in example rather than writing all the escaping from scratch. For brackets or new ordering, change the pattern; the analyzer does not need to change.

Regular expressions are matched using Python re.fullmatch after removing trailing CR/LF record delimiters. Ordinary spaces are not stripped from the entire line, so leading indentation must be allowed explicitly, for example with \s*. Source and severity values are trimmed. Messages are retained as captured; blank or whitespace-only messages fail. Inline regex flags such as (?i) can make literal labels case insensitive.

4 Map renamed or nested JSON fields

Consider this record:

```
{"event":{"at":1791369005000,"severity":40},"context":{"service":"db"},"msg":"Failed"}
```

Use this profile, available at examples/layouts/nested-json.json:

```
{
  "format": "ndjson",
  "fields": {
    "timestamp": ["event", "at"],
    "level": ["event", "severity"],
    "source": ["context", "service"],
    "message": "msg"
  },
  "timestamp_format": "unix_ms",
  "level_map": {"20": "INFO", "40": "ERROR", "50": "CRITICAL"}
}
```

A string mapping names a literal top-level key. For example, message: log.message means the actual key named log.message; it does not navigate an object. An array mapping navigates nested objects and arrays: ["events", 0, "at"] means the at field of the first object in events. Indexes must be nonnegative integers; booleans are rejected.

Unmapped normalized fields look up the original canonical key. Missing paths and null values are absent fields: source or level may use configured defaults, while missing timestamp or message is an error. Arrays and objects are not automatically stringified into messages or sources. Extra input fields remain ignored and are not retained as metadata.

Each physical line must be one JSON object. Use escaped \n inside a message for a logical newline; pretty-printing the object across physical lines is not supported. The filename may end in .json, .ndjson, or .log; the selected parser, not the extension, determines behavior.

5 Timestamp formats, severity aliases, and defaults

With a profile, timestamp_format has four modes:

- Omitted or iso: use Python datetime.fromisoformat on a string. Common date/time strings, T-separated ISO timestamps, fractional seconds, and offsets are accepted. Offsets are retained; naive dates remain naive. No timezone is inferred from the machine.
- A strptime format, such as %d/%m/%Y %H:%M:%S or %Y-%m-%d %H:%M:%S,%f: parse the explicitly declared layout. %z captures a numeric timezone offset. The exact directives available can depend on the platform. Use a year directive for meaningful real-log dates: omitted components use Python defaults, including year 1900 for a format without a year.
- unix: numeric seconds since the Unix epoch, converted to an aware UTC datetime.
- unix_ms: numeric milliseconds since the Unix epoch, also converted to UTC.

Numeric strings are allowed for explicit epoch modes. Booleans, nonfinite values, and out-of-range epochs fail. Invalid custom date directives are rejected when loading a profile, before attempting data records. Defaults do not fill missing timestamps. Native NDJSON, without a profile, retains its earlier Pydantic timestamp-coercion rules.

Profile and auto parsers uppercase and trim severities, then normalize WARN to WARNING, ERR to ERROR, and FATAL to CRITICAL. Other severities, including TRACE, must be mapped explicitly if wanted. Mapping keys ignore case and surrounding whitespace. Custom mappings can override built-in aliases; duplicate keys after normalization are rejected.

Numeric JSON severity codes require explicit mappings. For example, the sample's 40 to ERROR mapping is a profile choice, not a universal definition of code 40. Mapping values must be existing uppercase LogLevel values. No new severity enum was introduced.

A profile without defaults gets {"source":"unknown"}. Supplying a defaults object replaces that object; include source explicitly if you also want a level default:

```
{"source":"worker","level":"INFO"}
```

A level default is appropriate only when the application guarantees that severity for otherwise unlabeled records. Do not use it to conceal unknown severity fields. If a mapped record contains an unsupported severity, it fails rather than falling back to the default. Empty sources/messages and non-text values fail in configured parsers.

6 Automatic detection and its boundaries

--format auto tries only the following documented single-line grammars:

```
2026-10-07 10:30:00 INFO server Started
[2026-10-07 10:30:00] [INFO] [server] Started
[2026-10-07 10:30:00] [INFO] Started
2026-10-07T10:30:00Z INFO server Started
INFO | server | 2026-10-07T10:30:00 | Started
2026-10-07T10:30:00 | INFO | server | Started
```

Canonical one-object-per-line JSON with ISO timestamp strings is also recognized. For numeric JSON timestamps, use native NDJSON or an explicit epoch profile. Original-shaped text may include fractional seconds with a period or comma. The bracketed source is optional; absent source becomes unknown. Auto mode normalizes severities like a configured profile.

Detection succeeds only after a candidate both matches and validates all event fields. It then locks that parser for the stream. A later incompatible layout is a malformed record, not a reason to silently switch parsers. With --skip-invalid, bad leading records may be skipped until the first valid layout is found; later invalid records are skipped without unlocking the parser. Each stream has independent detection state.

Auto is not a confidence-scoring system or a schema mapper. Renamed JSON keys, arbitrary date styles, legacy syslog without complete dates, or different delimiter orders require a profile. Bracketed text can be intrinsically ambiguous: a message beginning with an extra bracketed token may look like a source field. Use an explicit profile for such data. There is no promise to identify every log format or to recover missing event semantics.

7 Implementation and design decisions

Extend adapters while preserving the analysis engine

Decision: add layouts.py for profile validation, normalization, and parser factories. The existing parse_lines loop still owns blank-line handling, physical line numbers, strict/tolerant behavior, and callbacks. errors.py owns LogParseError to avoid circular imports; parser re-exports the same class, preserving existing import paths.

Benefit: every new layout becomes the existing LogEntry, so selection, counts, error rate, exports, and reporting do not need changes. Regex patterns and severity maps are prepared once per stream. Reading remains lazy; no sample buffer or whole-file load is needed. JSON profiles are data and cannot import Python or evaluate expressions.

Trade-off: the model still requires timestamp, severity, source, and message; discarded metadata and multiline context are not recovered. Configured validation is deliberately stricter about text fields than the legacy model's empty-string allowance.

Alternatives: making the analyzer understand raw formats would duplicate parsing logic and break normalization. An abstract parser/plugin hierarchy may help with future complex segmentation, but two parser factories and one shared loop meet this need with less indirection. Executable Python profiles are flexible but introduce trust and distribution concerns. No plugin loader or general query engine was needed.

Named regex groups and explicit JSON paths

Decision: choose regex capture groups for configurable single-line text and literal-key/array-path mapping for JSON. Profiles are validated by the already-used Pydantic library.

Benefit: field ordering, punctuation, labels, timestamps, and nested JSON keys are explicit and reviewable. Users can save and reuse profiles without modifying source. Full matching avoids accepting an unrelated matching fragment of a longer record. Strict configuration keys expose typos rather than silently ignoring them.

Trade-off: regex syntax and JSON escaping require learning. Python re has no built-in matching timeout; poorly designed nested-quantifier patterns can be slow on adverse input. Profiles should be trusted and tested on representative lines. Literal-key/path syntax is intentionally simpler than full JSONPath and supports no wildcard selection.

Alternatives: delimiter-only splitting is easier but fails on labels, escaping, and variable layouts. A mini-template language is friendlier but needs its own escaping, grammar, and error handling. Grok offers reusable pattern vocabularies but introduces another dependency and compatibility surface. JSONPath or JMESPath is richer but adds syntax/dependencies where fixed paths suffice. Standard regex and explicit paths are a practical first extension; future UI-assisted profile creation could reduce the learning cost.

Explicit normalization and limited defaults

Decision: profiles declare date conversion and custom severity mappings. Built-in aliases cover only WARN, ERR, and FATAL. Source may default to unknown; level needs an explicit default. Timestamp and message are never supplied as invented defaults.

Benefit: source format differences are normalized before analysis. Numeric severity meaning is selected by the user rather than guessed. Required data cannot disappear behind permissive fallback logic. UTC is used for epoch values independent of local timezone.

Trade-off: many unfamiliar sources still need a profile. TRACE has no automatic mapping, because treating it as DEBUG loses a distinction. Custom strptime formats can fill omitted date components with Python defaults, so complete dates are recommended.

Alternatives: automatically guessing day/month ordering or numeric severity meanings is convenient but can yield valid-looking wrong events. Replacing all unknown levels with INFO conceals errors. Expanding the enum for every system complicates analyzer compatibility. Preserving raw metadata would be useful later but needs a deliberate model/output change. Explicit normalization keeps the current engine consistent.

Opt-in auto mode with per-stream locking

Decision: retain text as the default; make auto opt-in and limit it to known grammars. Lock only after a record validates. Keep state local to each parser factory.

Benefit: existing behavior remains predictable. Auto makes common logs usable immediately without silently accepting format changes halfway through a file. No separate scan or rewind is needed, so stdin and large files remain supported.

Trade-off: mixed-format files need preprocessing or an explicitly designed profile. The chosen layout is not added to the unchanged AnalysisResult schema. Ambiguous bracketed syntax cannot always reveal whether a token is a source or part of a message.

Alternatives: choosing a parser anew for every line accepts mixed logs but can silently change interpretation. Sampling and confidence scores may improve detection but need a buffer, thresholds, and ambiguity policy. Making auto the default would change legacy validation. Deterministic opt-in detection gives useful convenience with a smaller contract.

Fatal configuration errors and protected profiles

Decision: reject invalid regexes, groups, paths, date directives, duplicate normalized aliases, and unknown settings before record iteration. Configuration errors are separate from LogParseError and cannot be hidden by --skip-invalid. A report cannot overwrite its profile, including resolved-path and existing hardlink aliases.

Benefit: a typo cannot silently turn an entire file into skipped data, and the saved configuration remains reusable. Record failures keep the existing located diagnostics.

Trade-off: some permissive or partial configurations fail early. JSON profiles do not support fallback lists of candidate fields or arbitrary transformations.

Alternatives: treating config failures as malformed records can make empty reports look successful. Allowing dynamic transformation code is more flexible but moves beyond safe declarative profiles. Adding only one broad exception catch would hide programmer errors. Separate validation and record errors keep failure handling understandable.

8 Validation and changed files

The development suite has 128 passing tests: the original 58 plus 70 layout tests. Existing text/NDJSON, filtering, report formats, streaming, atomic output, and resource handling tests remain. New coverage includes regex field order, defaults, full matching, split date/time, custom dates and aliases, invalid profiles/paths, epoch conversions, nested arrays and literal dotted keys, source fallback, missing/wrong fields, known auto grammars, locking and skipping, compatible JSON aliases, laziness and located errors, shipped profiles, stdin, configuration conflicts, and overwrite protection.

Changed files: parser.py dispatches native/configured/auto parsers; cli.py adds --parser-config and extended --format choices; MANIFEST.in includes JSON profiles; README, CHANGELOG, architecture notes, and CI smoke commands describe/exercise the new behavior. Added files: layouts.py, errors.py, test_layouts.py, bracketed/custom-pipe/ nested samples, two example profiles, and this guide. Existing analysis and report schemas were not changed; no new runtime dependency was introduced.

The same 128 tests passed against the rebuilt wheel installed in an isolated Python 3.14.4 environment outside the repository, with copied tests/examples and no src directory. The installed custom-profile command produced the expected 50% ERROR rate, and pip check found no broken requirements. The wheel was built offline with the available setuptools backend. CI configuration now also exercises auto and custom-profile commands; remote CI and other Python versions have not been run during this update. The package remains the unpublished 0.1.0 local milestone, with the layout extension recorded under Unreleased in CHANGELOG.

9 Remaining work and references

Useful next improvements include profile authoring assistance, additional named presets, structured run metadata describing the selected layout, bounded line/regex execution, and real-world fixture collections. Whole-file JSON arrays and multiline records need streaming event segmentation rather than extending the current physical-line loop. Legacy syslog year/timezone resolution needs explicit context. Plugin loading, automatic schema inference, and arbitrary code transformations remain intentionally unimplemented.

Official API references used for this implementation:

- Python named regex groups and full matching: https://docs.python.org/3/library/re.html
- Python date parsing and ISO conversion: https://docs.python.org/3/library/datetime.html
- Pydantic model validators: https://docs.pydantic.dev/latest/concepts/validators/

For installation, all CLI options, output schemas, and original analysis semantics, see README.md. For initial MVP decisions, see docs/implementation-report.md and its original PDF. This guide is the current supplement for configurable layouts.